

# Write-Up

## CMPT310 - D200

### Group 20

|                   |               |
|-------------------|---------------|
| Abraham Mormontoy | aem19@sfu.ca  |
| Aly Soliman       | asa516@sfu.ca |
| Karan Passi       | kpa77@sfu.ca  |
| Viktor Zepeda     | vza12@sfu.ca  |

## System Explanation

Our system takes as an input images of dogs, and outputs the top-3 most likely breeds. We use transfer learning on the pretrained convolutional neural network ResNet-18 to train our model by freezing the network's base layers so they don't influence our learning, as well as changing the final layer to map features to our 50 classes.

## AI Pipeline

Our pipeline follows 5 steps:

### Data collection

We first gather [Stanford's Dog Dataset](#) which contains 120 folders from various dog breeds, of which we only use 50 to keep variety and not prediction accuracy.

### Preprocessing

The images go through a pruning and cleaning process to get exactly 150 images per class and get rid of the original name prefixes in the directories. Then a series of transformations are applied to resize each image, randomly flip and rotate it. The images are then converted to PyTorch tensors and normalized to match ResNet-18's pixel distribution.

### Transfer learning

Since we are using a pretrained convolutional neural network, we have to freeze the network's base layers and then replace the final output layer with our own. This approach speeds up the process of training, since we only have to worry about the final layer. An added benefit of using this approach is that it avoids overfitting to our data, which would have posed a significant challenge had we trained our own CNN from scratch, since each class only has 150 images.

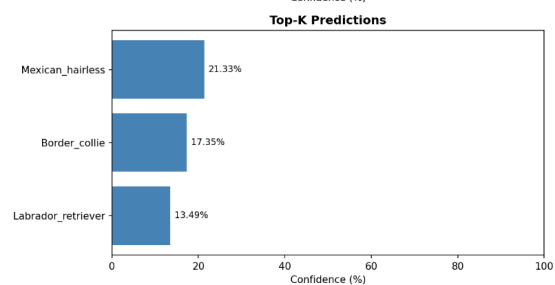
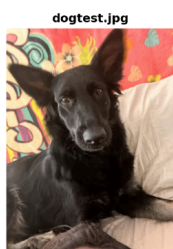
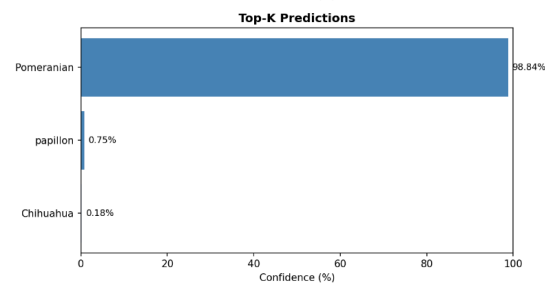
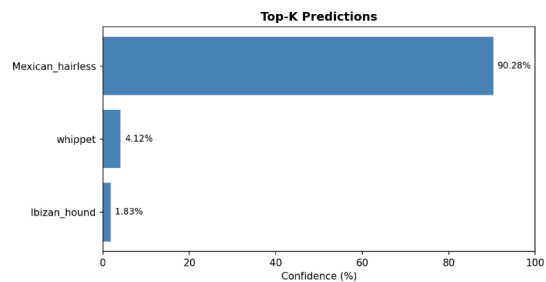
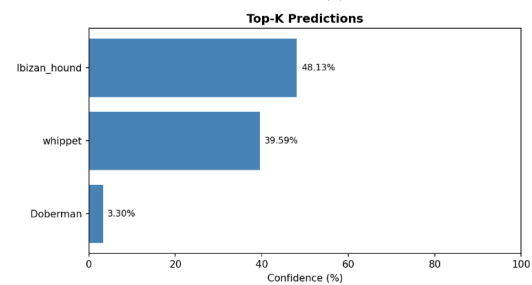
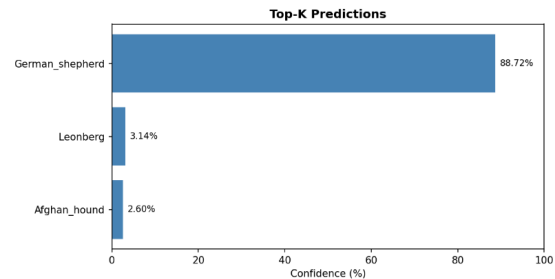
ResNet-18's original final layer performs `Linear(512, 1000)`, which maps 512 feature values to 1,000 ImageNet classes. We replaced it with `Linear(512, 50)` to match our 50-breed output. This new layer is randomly initialised and is the only layer trained during our experiments.

### Training

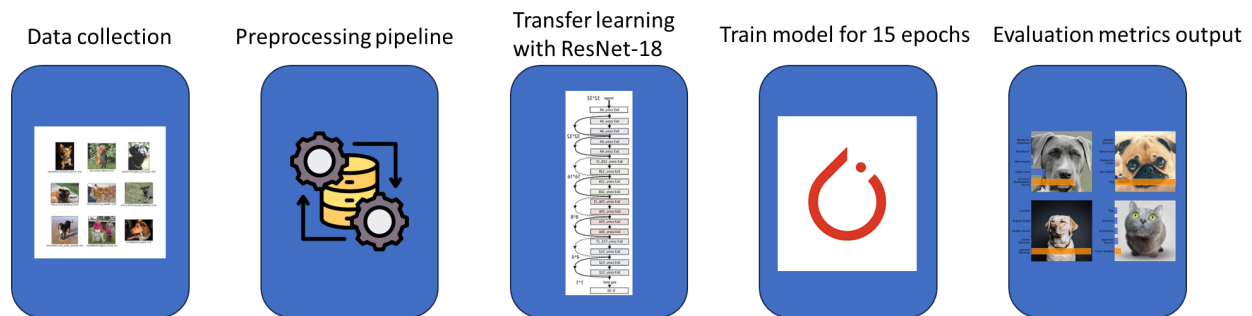
We used a 72/25 train/test split for better validation. We standardized our training to 15 epochs on a CUDA-powered GPU, as our testing showed that the model converges within 10-to-15 epochs. During each forward pass, images move through the network to generate predictions for all 50 classes. We compare these to the labels using the loss function. In the backward pass, gradients are computed via backpropagation so the optimizer can update the model's parameters. The use of transfer learning helped achieve faster training.

## Output

For the final stage we produce three plots: a bar chart with the prediction accuracy of the 50 breeds, a plot with the train and test loss, and another bar chart with the top 3 predictions for our hand chosen dogs, German Shepherd, Pomeranian, Mexican Hairless and Italian Greyhound.



## Diagrams, model architecture, and flowcharts



## Detailed Preprocessing pipeline

|                             |                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Resize(224, 224)</b>     | This scales every image to exactly $224 \times 224$ pixels. This is done because ResNet-18 requires this fixed input size because its architecture ends with a fixed-size fully-connected layer.                                                                                                                                              |
| <b>RandomHorizontalFlip</b> | mirrors the image left-to-right with a 50% probability. A dog facing left and a dog facing right are the same breed, so this step is done to teach the model that horizontal orientation is not a discriminating feature.                                                                                                                     |
| <b>RandomRotation(15)</b>   | Randomly rotates the image by a uniformly sampled angle between $-15^\circ$ and $+15^\circ$ . Dogs in photos are rarely perfectly upright; this makes the model more robust to slight tilts in the input image.                                                                                                                               |
| <b>ToTensor</b>             | PIL Image $\rightarrow$ torch.FloatTensor of shape $[3, 224, 224]$ . Pixel values are rescaled from the integer range $[0, 255]$ to floats in $[0.0, 1.0]$ .                                                                                                                                                                                  |
| <b>Normalize(mean, std)</b> | Shifts and scales each colour channel using ImageNet statistics (mean = $[0.485, 0.456, 0.406]$ , std = $[0.229, 0.224, 0.225]$ ). This is required because ResNet-18's pretrained weights were optimised for this exact pixel distribution. Skipping normalisation would cause the frozen feature layers to produce meaningless activations. |

## Limitations

Our model does well on the images from our hand picked dataset of 50 dogs but performs less accurately on random dog images from the internet or that we've taken ourselves. This could potentially be solved by training with more classes

## Feature Table

| Description                  | Platform | Completeness | Code   | Author(s)                | Notes                                                                                                               |
|------------------------------|----------|--------------|--------|--------------------------|---------------------------------------------------------------------------------------------------------------------|
| Image Preprocessing          | Local    | 5            | Python | Abraham/<br>Viktor / Aly | Cleaned dataset.<br>Original code                                                                                   |
| CNN classifier               | Local    | 5            | Python | Aly / Abraham            | Used the ResNet-18 model and froze its base layers                                                                  |
| Train/test loss plots        | Local    | 5            | Python | Viktor                   | While training we keep a log of the losses and accuracy and then we plot that                                       |
| Predicting a single image    | Local    | 4            | Python | Abraham                  | The accuracy on predicting a single image isn't too high but the actual breed appears in the top 3 most of the time |
| Accuracy plots for 50 breeds | Local    | 4.5          | Python | Aly / Viktor             | After training we mimic Stanford's accuracy plo                                                                     |

## External Tools and Libraries

### Frameworks and ML libraries used:

- PyTorch: Framework for training the CNN model.
- Torchvision: Image transforms, dataset loaders, and pretrained ResNet-18 weights for transfer learning.
- Scikit-learn: Logistic Regression baseline, train-test split, scaling, PCA, and evaluation metrics.

### Image processing and visualization:

- OpenCV: Image loading and resizing/preprocessing.

- Pillow: Image reading for prediction workflows.
- NumPy: Numerical array operations.
- Matplotlib: Training and prediction plots.
- Pandas: Training logs and CSV metrics output.

### Dataset and data source:

- Local dog breed dataset organized by class folder one folder per breed with the breed in the name

### Open-source:

- Reused open-source ecosystems: PyTorch, Torchvision, Scikit-learn, OpenCV, Pillow, NumPy, Matplotlib, Pandas.
- Reused pretrained resource: Torchvision ResNet-18 pretrained weights

### Platform dependencies and installation:

- Python version: Python 3.10+
- More acceleration: CUDA-compatible GPU for faster CNN training, this is optional as it works with CPU too
- Installation command: `pip install torch torchvision pandas matplotlib pillow numpy scikit-learn opencv-python`
- Dataset Requirement: dog images breeds have to be placed in a folder named dataset inside of both logistic regression and CNN

## Project Directory

